Mini relatório: Broker MQTT (IoT)

Projeto de monitoramento de CO2 · ESP32 + SCD30 · PI III

1. O que foi feito

Ambiente e broker

- Instalação do **Docker Desktop** (com WSL 2).
- Criação de um broker **Eclipse Mosquitto** rodando em container, iniciado com `docker compose up -d`.
- Arquivos criados: `docker-compose.yml` e `config/mosquitto.conf`.
- Container: `meu_broker_mqtt`.

Portas do broker

- **1883**: MQTT padrão (o ESP32 publica e o backend Java assina por aqui).
- **9001**: MQTT sobre WebSocket (disponível caso o front-end precise).
- `allow_anonymous true`: não é necessário usuário e senha para conectar.

Contratos (já no repositório, em docs/contratos)

- **CT1 - publishReading**: tópico `/sensor/data`, com o JSON contendo `deviceId, co2, temperature, humidity` e `measuredAt` (obrigatório).
- **CT3 - sensorStatus**: tópico `co2nect/{deviceId}/status` (sensor online/offline).

Exemplo de payload do CT1:

```
{
  "deviceId": "ESP32-SALA-01",
  "co2": 850,
  "temperature": 24.5,
  "humidity": 45.2,
  "measuredAt": "2026-10-13T19:20:00"
}
```

2. Como testar o broker

Quem quiser testar o broker precisa baixar o Docker Desktop (docker.com). Ele é necessário para rodar o Mosquitto em container.

1. Instalar o Docker Desktop e deixá-lo aberto e rodando.
2. Clonar o repositório e abrir o terminal na pasta do broker (onde está o `docker-compose.yml`).
3. Executar `docker compose up -d`.
4. Conferir com `docker ps`: deve aparecer o container `meu_broker_mqtt`.

Dados de conexão

- Host: `localhost` (ou o IP da máquina que estiver rodando o broker).
- Porta: `1883`.
- Autenticação: nenhuma.

Teste manual (sem o ESP32)

- Abrir o **MQTT Explorer** (mqtt-explorer.com) e conectar em localhost, porta 1883.

- Publicar o JSON de exemplo do CT1 no tópico `/sensor/data` para simular o sensor.
- Usar o mesmo recurso para conferir se o backend Java está recebendo a mensagem.

3. O que a equipe de Banco e Conexão precisa fazer

Adicionar o PostgreSQL ao docker-compose do broker

No broker foi colocado apenas o Mosquitto. O PostgreSQL deve ser adicionado como um novo serviço no mesmo `docker-compose.yml`, para que um único comando suba os dois. Exemplo (ajustar nome, usuário e senha):

```
postgres:
  image: postgres:16
  container_name: meu_postgres
  environment:
    POSTGRES_DB: co2nnect
    POSTGRES_USER: usuario
    POSTGRES_PASSWORD: senha
  ports:
    - "5432:5432"
  volumes:
    - ./pgdata:/var/lib/postgresql/data
  restart: unless-stopped
```

Esse bloco entra dentro de `services:`, ao lado do `mosquitto`.

Quem faz o banco de dados (PostgreSQL)

- Criar a tabela de leituras com base no CT1: `device_id` (texto), `co2` (inteiro), `temperature` (decimal), `humidity` (decimal) e `measured_at` (data e hora, obrigatório).
- Se quiser guardar o status do sensor (CT3), criar também uma estrutura para ele.

Quem faz a conexão (Java)

- Conectar ao broker usando um cliente MQTT para Java (por exemplo, Eclipse Paho).
- Assinar (subscribe) o tópico `/sensor/data` e o de status `co2nect/+/status` (o `+` representa qualquer deviceId).
- Criar um DTO que mapeie o JSON do CT1 e gravar os dados no PostgreSQL.
- Ler os contratos em `docs/contratos` para conferir campos e formatos.